

Real-Time Services in Linux

Real-Time Operating Systems

Paulo Pedreiras

DETI / UA / IT

Sept/2026

Agenda

- 1 Part I - Preliminaries
- 2 Part II - Overview of Linux Real-Time Services Scheduling
- 3 Part III - Using Linux Real-Time Services
- 4 Part IV - Evaluating RT Performance

Part I - Preliminaries

Embedded Linux and Real-Time (1)

- Linux and other open-source software are increasingly used for developing Embedded Systems.
- Many of these applications have real-time requirements.
- Ideally we would like to:
 - Have the advantages of Linux: HW support, low cost, modularity, openness, ...
 - And meet deadlines!



?



Embedded Linux and Real-Time (2)

- Linux was developed as generic desktop and server time-sharing OS:
 - **Main Objectives:** optimize throughput, improve average utilization of resources (CPU, memory, I/O), openness, ...
 - Timeliness was (and generically is) not a main issue for GPOS.
- Conversely, **dedicated Real-Time Operating Systems (RTOS)** are engineered for temporal **determinism**, often at expenses of throughput, flexibility, openness, HW abstraction, ...
- In fact, **throughput, flexibility, and openness often conflict with determinism.**
 - Optimizing for one often impacts negatively on the other.

Linux and Real-Time Approaches

Improving the real-time performance of Linux

Two ways to handle this conflict:

Approach 1: **Modify the Linux kernel to improve its temporal behavior**

- Bound the latency of syscalls, introduce fine-grain preemption, improve timer resolution, create specific services for real-time, proper scheduling, etc.
- Over time, many of these features have been added to the mainline Linux kernel (from the PREEMPT_RT project).

Approach 2: **Hypervisor approach: add a RTOS “under” Linux**

- Linux becomes a background task of the real-time RTOS tasks.
- Approach followed by many Real-Time Linux RTOSs (e.g., RTAI, Xenomai).

Understanding Latency

Hints on Linux kernel latency sources

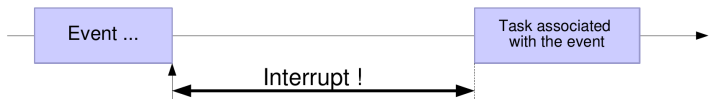
A typical event sequence in a real-time application is:

- 1 An event (can be a timer event) causes a CPU interrupt.
- 2 The corresponding ISR is executed, activating a user-space task associated with the event.
- 3 Eventually this task is executed and completes, thus closing the reaction to the event.

Time elapsed between an event and the corresponding task activation is the **latency**.

Objectives:

- Bound latency
- Reduce latency as much as possible!

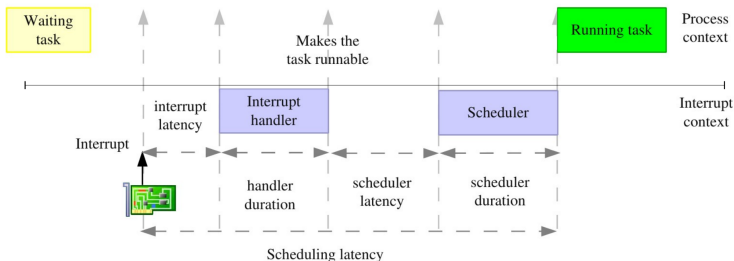


Linux Kernel Latency Components

Hints on Linux kernel latency sources

Kernel Latency Equation

kernel latency = interrupt latency + handler duration + scheduler latency + scheduler duration



Source of Interrupt Latency

Hints on Linux kernel latency sources

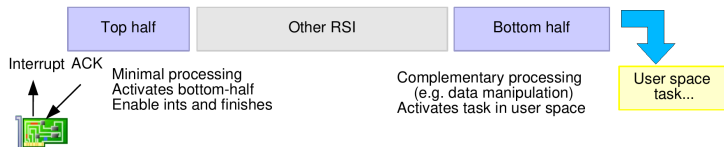
Interrupt latency sources:

- Linux kernel (including device drivers) disables interrupts for certain operations.
- Interrupts can interrupt other interrupts (nesting).

Interrupt handlers are split in two parts (top/bottom; fast/slow):

Top half: Minimal processing, acknowledges interrupt, activates bottom-half, enables interrupts and finishes.

Bottom half: Complementary processing (e.g., data manipulation), activates task in user space.



Understanding Preemption (1)

Hints on Linux kernel latency sources

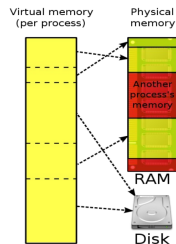
Linux kernel is preemptive, but **not fully!** Syscalls have variable duration and often limited preemption.

Other Non-Deterministic Mechanisms

Hints on Linux kernel latency sources

Additional sources of latency and jitter affecting RT tasks in Linux:

- Scheduler latency
- Virtual memory usage (data on disk takes longer than RAM)
- Cache misses
- Non-deterministic C library functions
- Priority inversion (due to shared resources)
- Interrupt prioritization



The PREEMPT_RT Project

- Developed by Ingo Molnar, Thomas Gleixner, and Steven Rostedt (<https://rt.wiki.kernel.org>).
- Maintained at: <https://wiki.linuxfoundation.org/realtime/start>
- Documentation:
<https://wiki.linuxfoundation.org/realtime/documentation/start>
- **Objective:** Gradually improve real-time behavior of the Linux kernel and mainlining those changes.
- Most improvements are already integrated into the mainline kernel.

Improvements in the Mainline Kernel

Mainline kernel features integrated from PREEMPT_RT:

- Fixed-priority scheduler
- Improved kernel preemption
- POSIX real-time API support improvements
- Mutexes with Priority Inheritance Protocol (PIP) support
- High resolution timers
- Threaded interrupts
- `sched_deadline`, EDF scheduling with CBS

- A few Linux distributions providing pre-patched kernels:
 - **Real-time Ubuntu, from 22.04 LTS**
 - Free for private use.
 - Integrates out-of-tree PREEMPT_RT partial patches for x86 and Arm architectures.
 - **Real-time Ubuntu 26.04 LTS** PREEMPT_RT patches have been fully upstreamed. The real-time kernel is freely available for everyone from the 26.04 release onwards.
 - Just type `$sudo apt install ubuntu-realtime` at the command prompt
 - See: <https://ubuntu.com/real-time>

Preemption Control Options

Linux kernel supports different preemption levels configured at compile-time:

- `CONFIG_PREEMPT_NONE`
- `CONFIG_PREEMPT_VOLUNTARY`
- `CONFIG_PREEMPT`

`PREEMPT_RT` patch provides additional options:

- Preemptible Kernel or Basic RT (`PREEMPT_RT_B`)
- Fully Preemptible Kernel (`PREEMPT_RT_FULL`)

1st Option: No Forced Preemption

`CONFIG_PREEMPT_NONE`

- Kernel code (interrupts, exceptions, system calls) is never preempted.
- Common default configuration in throughput-focused distributions.
- Maximizes throughput for compute-heavy workloads.
- Minimizes context switches and associated overheads.

2nd Option: Voluntary Kernel Preemption

`CONFIG_PREEMPT_VOLUNTARY`

- Kernel code is preemptible at specific explicit rescheduling points.
- Useful for desktop environments to improve user-perceived responsiveness.
- Low overall impact on throughput.

3rd Option: Preemptible Kernel

CONFIG_PREEMPT

- Most of the kernel becomes preemptible.
- Dispatched processes can start execution before completion of pending system calls.
- Non-preemptible critical sections remain protected by spinlocks.
- Best option for Embedded Systems with RT requirements (without PREEMPT_RT patch).

PREEMPT RT patch modes

PREEMPT_RT modes

CONFIG_PREEMPT_RT: Basic Real-Time

- Applies the foundational layers of the RT patch.
- Forces hardware interrupt handlers (IRQs) and SoftIRQs to execute as schedulable kernel threads rather than in a strict interrupt context.
- It does not convert standard kernel spinlocks into sleepable mutexes. Whenever the kernel enters a critical section protected by a spinlock, preemption is still disabled.

CONFIG_PREEMPT_RT_FULL: Fully Preemptible Kernel

- Aims at production environments requiring hard real-time constraints (e.g., industrial robotics, audio processing, aviation).
- It includes everything from Basic RT, and additionally converts almost all kernel spinlocks (spinlock_t), read-write locks, and standard mutexes into preemptible, sleepable rt_mutexes.
- Prevents "priority inversion" via the the newly implemented rt_mutexes, which feature strict priority inheritance.

Checking Kernel Configuration

Check preemption configuration of the running kernel:

```
$ cat /boot/config-$(uname -r) | grep PREEMPT
# CONFIG_PREEMPT_NONE is not set
# CONFIG_PREEMPT_VOLUNTARY is not set
CONFIG_PREEMPT=y
CONFIG_PREEMPT_COUNT=y
CONFIG_PREEMPTION=y
CONFIG_PREEMPT_DYNAMIC=y
CONFIG_PREEMPT_RCU=y
CONFIG_HAVE_PREEMPT_DYNAMIC=y
```

Part II - Overview of Linux Real-Time Services Scheduling

Linux Scheduling Classes: Time-Sharing

- The default scheduling class is time-sharing: `SCHED_OTHER`.
- Implemented by the **Completely Fair Scheduler (CFS)**.
- No fixed time slices; dynamic CPU allocation based on `vruntime`:

$$\text{vruntime} \approx \text{actual_runtime} \times \text{weight normalization}$$

- Priority controlled via “nice” values (−20 highest to 19 lowest).
- **Non-deterministic:** Poor real-time performance.

Fixed-Priority Real-Time Classes

- Two fixed-priority real-time classes: `SCHED_FIFO` and `SCHED_RR`.
- Highest priority ready task always gets the CPU.
- Static priorities ranging from 1 (lowest) to 99 (highest).
- `SCHED_FIFO`: Same priority tasks execute in arrival order.
- `SCHED_RR`: Round-robin time-slicing among same-priority tasks.
- Significantly improved determinism (latencies down to tens/hundreds of μs).

Managing RT Priorities via CLI

Assign policy and priority using chrt:

```
$ chrt -f 99 ./myprog      # Set FIFO policy, priority 99
$ chrt -p 877145          # Query process attributes
pid 877145's current scheduling policy: SCHED_FIFO
pid 877145's current scheduling priority: 25
```

Using top or htop (top -H for threads):

- RT priorities are displayed as negative values with an offset of 1 (e.g., RT priority 25 shows as -26).

- Dynamic priority scheduling implementing EDF + CBS.
- Takes precedence over FIFO and RR classes
- Takes the form of a contract with the kernel, with parameters:
 - **Runtime:** Max execution time per period.
 - **Deadline:** Relative deadline.
 - **Period:** Activation period ($\text{Runtime} \leq \text{Deadline} \leq \text{Period}$).

```
# Set SCHED_DEADLINE (Runtime=1ms, Deadline=5ms, Period=50ms)
$ chrt -d --sched-runtime 1000000 --sched-deadline 5000000 \
    --sched-period 50000000 0 ./dummyTask &
```

Limiting RT Utilization

- Linux allows bounding total CPU utilization for RT tasks to prevent starvation of non-RT tasks.
- Configured via `/proc/sys/kernel/`:
 - `sched_rt_period_us` (default 1 000 000 μs)
 - `sched_rt_runtime_us` (default 950 000 $\mu s \Rightarrow 95\%$)
- Bounding value `-1` sets runtime = period (disables throttling limit).

Programmatic Task Control & Priority Notes (1/2)

Retrieving task info programmatically:

- `gettid()`: Returns thread ID.
- `sched_getscheduler(0)`: Returns scheduling policy.
- `sched_getparam(0, &parm)`: Returns RT priority.

Kernel vs. User Space Priorities:

- Mappings between user-space priorities (1–99) and internal kernel representation can cause confusion.
- Always use POSIX APIs (`sched_get_priority_max/min`) for portable code.

About priorities in Linux

- The issue of process/thread priorities is confusing, because there is (or should be) a separation between how priorities show up at user space and how they are handled internally by the kernel.
- To complicate things further, the RT services are an addition that had significant implications on the kernel, resulting in some inconsistencies in how a few attributes are handled in different parts of the OS.
- Interested students can find an interesting discussion on this topic in the following Stack overflow forum discussion - Which real-time priority is the highest priority in Linux:

<https://stackoverflow.com/questions/8887531/>

[which-real-time-priority-is-the-highest-priority-in-linux](https://stackoverflow.com/questions/8887531/which-real-time-priority-is-the-highest-priority-in-linux)

Part III - Using Linux Real-Time Services

Development and Compilation

- No special tools are required.
 - POSIX real-time extensions are integrated into standard C libraries (glibc).
- Compile/link using `-lrt`:

E.g. `$gcc -o myprog myprog.c -lrt -pthread`

- API documentation:
 - <https://wiki.linuxfoundation.org/realtime/documentation/start>
 - `$man functionname`

Setting Scheduling Classes Programmatically

At **process level**, using `sched_setscheduler`:

```
int sched_setscheduler(pid_t pid, int policy,  
const struct sched_param *param);
```

- pid: pid of the thread. Calling `thread` if 0
- policy: `SCHED_OTHER`, `SCHED_FIFO`, `SCHED_RR`, etc.
- param: structure that includes priority

For **each thread**, at thread creation time:

```
struct sched_param parm;  
pthread_attr_t attr;  
  
pthread_attr_init(&attr);  
pthread_attr_setinheritsched(&attr, PTHREAD_EXPLICIT_SCHED);  
pthread_attr_setschedpolicy(&attr, SCHED_FIFO);  
parm.sched_priority = 42;  
pthread_attr_setschedparam(&attr, &parm);  
pthread_create(&thread, &attr, task_func, NULL);
```

Note: by default threads inherit the RT class and priority of the main thread -
pthreads API defaults the thread creation attribute to
`PTHREAD_INHERIT_SCHED`

Creating and Managing Threads

Standard POSIX thread calls work as usual:

- **Create:** `pthread_create(&thread, &attr, routine, arg);`
- **Exit:** `pthread_exit(value_ptr);`
- **Join:** `pthread_join(thread, &value_ptr);`

Memory Locking & Prefaulting

To prevent paging and page-fault latency, lock physical memory:

```
#include <sys/mman.h>

// Lock all current and future allocated pages in RAM
mlockall(MCL_CURRENT | MCL_FUTURE);
```

Related syscalls: `munlockall`, `mlock`, `munlock`.

Prefaulting:

- Touch stack and pre-allocate heap buffers prior to entering the real-time loop to avoid dynamic allocation overheads during runtime.

Mutexes & Priority Inheritance

Mutex: allow implementing mutual exclusion between threads of the same process with (optional) Priority Inheritance

- Creation and elimination

```
pthread_mutex_init(pthread_mutex_t *mutex,  
const pthread_mutexattr_t *mutexattr);  
pthread_mutex_destroy(pthread_mutex_t *mutex);
```

- Lock/unlock

```
pthread_mutex_lock(pthread_mutex_t *mutex);  
pthread_mutex_unlock(pthread_mutex_t *mutex);
```

- For using priority inheritance:

```
pthread_mutexattr_t attr;  
pthread_mutexattr_init (&attr);  
pthread_mutexattr_getprotocol (&attr,  
PTHREAD_PRIO_INHERIT);
```

Condition Variables

Condition variables provide synchronized notification avoiding signals, which have several issues.

Declaration and initialization illustration:

```
#include <pthread.h>
...
pthread_condattr_t cattr;
pthread_cond_t cond;
...
pthread_condattr_init(&cattr);

/* Indicates that cond var can be accessed by threads of multiple */
/* processes. Set only if appropriate.                               */
/* pthread_condattr_setpshared(&cattr, PTHREAD_PROCESS_SHARED); */

pthread_cond_init(&cond, &cattr);
```

Condition Variables

Receiver code snippet:

```
...  
pthread_mutex_lock(&lock);  
pthread_cond_wait(&cond, &lock);  
/* Wait for sender */  
DO_PROCESSING  
pthread_mutex_unlock(&lock);  
...
```

Sender code snippet:

```
...  
pthread_mutex_lock(&lock);  
DO_PROCESSING  
pthread_cond_broadcast(&cond);  
pthread_mutex_unlock(&lock);  
...
```

Periodic Task Execution

Most Real-Time applications have periodic tasks.

- The best way to create them in Linux is using the `clock_nanosleep` syscall
- Use `clock_nanosleep()` with `CLOCK_MONOTONIC` and absolute timing (`TIMER_ABSTIME`):

```
struct timespec ts;
clock_gettime(CLOCK_MONOTONIC, &ts);

while(1) {
    add_period(&ts, PERIOD_NS); // Increment absolute target time
    clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &ts, NULL);

    /* Perform Periodic Task Processing */
    ...
}
```

A note on timers:

- Linux has different timers (or `clock_id`) that can be used by time-related syscalls such as `timer_create()` and `clock_nanosleep()`, namely:
 - `CLOCK_MONOTONIC`
 - `CLOCK_REALTIME`
 - `CLOCK_PROCESS_CPUTIME_ID`
- Despite its name, `CLOCK_REALTIME` should not be used in real-time applications. Real-time in this case refers to the “calendar time” and this clock can be modified during runtime (by the user, by NTP, ...), disrupting real-time activities
- `CLOCK_MONOTONIC` is the right choice

CPU Affinity

- Isolating activity on CPUs can increase determinism by controlling which tasks interfere with each other (on the CPU)
- Note that SMP systems usually have multiple CPUs sharing caches (at least at some levels), thus tasks in different cores can still interfere. Buses and other shared resources are also potential additional sources of interference ...

The `sched_setaffinity()` syscall allows to define in which CPU(s) tasks may execute.

```
#define _GNU_SOURCE
#include <sched.h>

cpu_set_t set;
CPU_ZERO(&set);
CPU_SET(1, &set); // Bind to CPU 1.
CPU_SET(2, &set); // Make it run also in CPU 2, if you want
...
sched_setaffinity(0, sizeof(cpu_set_t), &set);
```

- Hardware interrupts can also be routed to specific CPUs via affinity masks
 - The default CPU affinity for routing hardware interrupts when new interrupt handlers are registered can be viewed/set in:
 - `/proc/irq/default_smp_affinity`

The CPU affinity for routing hardware interrupts for already registered interrupt handlers can be viewed/set in:

 - `/proc/irq/irq-number/effective_affinity`
 - `/proc/irq/irq-number/smp_affinity`- CPUs can be isolated from interrupts and scheduler at boot. See:
<https://www.kernel.org/doc/html/latest/admin-guide/kernel-parameters.html?highlight=isolcpu>

Part IV - Evaluating RT Performance

Profiling with perf

perf

- perf is a tool that allows counting various types of hardware and software events.
- Examples include: CPU cycles, page faults, cache misses, context switches and scheduling events
- It can help to identify the causes of eventual performance issues experimented by real-time tasks by identifying relevant events

Usage:

```
perf [--version] [--help] [OPTIONS] COMMAND [ARGS]
```

Most important commands: `perf -help` and `perf -help jcommand;`

Part of Linux tools package. May not be installed by default.

Profiling with perf

perf (cont). A few usage examples:

- List all supported events:
 - `perf list`
- Run a command and get counter stats (stat), from all CPUs (-a), specific events (-e). Command is "dd"
 - `perf stat -a -e cpu-cycles -e page-faults dd if=/dev/zero of=/dev/null count=1000000`
- Get stats for (running) process with PID 123456. Can aggregate more than one process. Just add the PIDs after "-p", separated with a ",",
 - `perf stat -p 123456`
- View the top symbols causing cache misses on the first CPU (perf top -C 0), with perf pinned to the second CPU (taskset 2):
 - `taskset 2 perf top -C 0 -e cache-misses`
- Many good tutorials, with helpful examples. E.g.:
<https://perf.wiki.kernel.org/index.php/Tutorial>

Kernel Tracing with ftrace & trace-cmd

Ftrace: the Linux Tracing Infrastructure

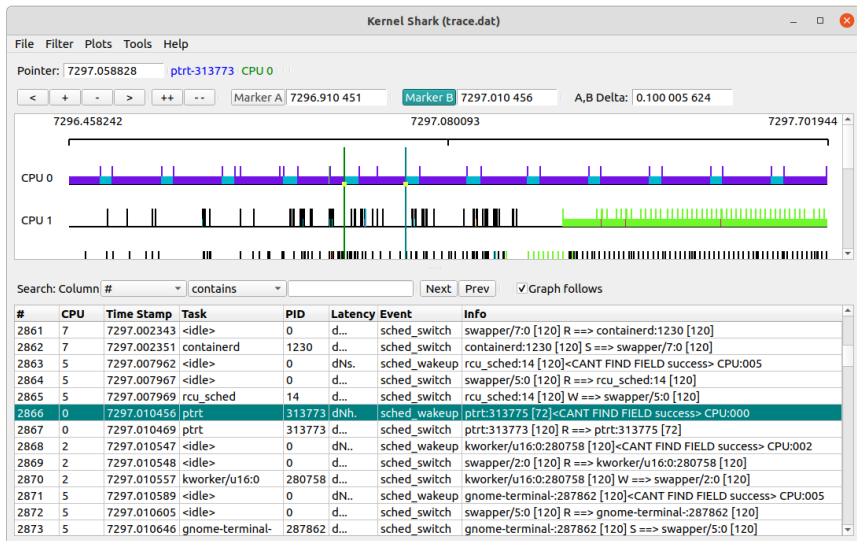
- Ftrace (Function Tracer) is a tracing framework for the Linux kernel
- Added to the kernel back in 2008 and evolving over time since then
- Allows the user to observe and record the execution flow of kernel functions
 - Allows to find out **which kernel functions are called** by a given user space application.
 - Allows to **measure execution time** and find out bottlenecks and **performance issues**.
 - Allows to identify hangs in kernel space.
 - And many more ...
- Direct use of Ftrace is very powerful but also complex
 - There are tools that simplify usage, such as trace-cmd and perf.
 - There are also extremely convenient graphical tools that allow to view and analyze trace data (e.g. kernelshark and Trace Compass).

Kernel Tracing with ftrace & trace-cmd

Ftrace (cont): A few simple examples:

- List all supported events:
 - `trace-cmd list`
- Record scheduling wakeup and switch events system-wide while running a periodic RT process (RT prio 27, named “xpto”):
 - `sudo trace-cmd record -e sched:sched_wakeup -e sched:sched_switch ./ptrt xpto 27`
- Recorded events are recorded in a binary format file (trace.dat).
Readable/textual data can be obtained via the report command:
 - `trace-cmd report`
- But it is much more convenient to see the info graphically:
 - `kernelshark`

Kernel Tracing with ftrace & kernelshark



Stress Testing with stress-ng

stress-ng

- Inject controlled synthetic workload to verify real-time latencies under stress
- Highly customizable (how many CPUs to use, different types of tests, etc.)

Additional techniques to improve RT performance

There are several other approaches to improve the RT performance of real-time applications in Linux, e.g.:

- Evaluate the performance of device-drivers
 - Some device-drivers have code that has poor RT performance. Such device-drivers (and consequently such hardware) should be avoided
- Tune memory allocation:
 - glibc's `malloc()` function can be tuned via the `mallopt()` syscall
 - E.g. disable memory mapping for memory allocation and disable heap trimming:
`mallopt(M_MMAP_MAX, 0); mallopt(M_TRIM_THRESHOLD, -1)`
 - After issuing `mlockall()` force stack/heap prefaulting, i.e. allocate (and access) all stack and heap memory that the application may need before releasing the tasks

Main Aspects Impacting RT Performance

Summary Best Practices

- **Schedulers:** Use `SCHED_FIFO` or `SCHED_RR`.
- **CPU Affinity:** Isolate cores to reduce task interference.
- **Memory:** Call `mlockall()` and pre-fault stack/heap.
- **Clocks:** Use `CLOCK_MONOTONIC` with absolute sleep timing.
- **Synchronization:** Use Priority Inheritance Mutexes and Condition Variables.
- **Validation:** Benchmark under stress using `perf`, `trace-cmd`, and `stress-ng`.

Further Reading & References

- Real-Time Linux Wiki:
<https://wiki.linuxfoundation.org/realtime/start>
- Reghenzani et al., *"The Real-Time Linux Kernel: A Survey on PREEMPT_RT"*, ACM Computing Surveys, 2019.
- OSADL Real-Time Linux Project: <https://www.osadl.org>
- Linux Ftrace documentation:
<https://www.kernel.org/doc/html/latest/trace/ftrace.html>
- Ubuntu Real-Time Linux Series:
<https://ubuntu.com/blog/what-is-real-time-linux-i>

Note: Some of these slides were adapted from “Real-time in embedded Linux systems”, by M. Opdenacker, T. Petazzoni e G. Chanteperrin